

Approach — Conversational SHL Assessment Recommender

SHL Labs AI Intern take-home. ~2 pages. All metrics are produced by the eval harness in `eval/`.

Problem framing

The task is a **grounded, multi-turn recommendation** problem, not a search box. The agent must move a user from vague intent to a shortlist of 1–10 **real** SHL assessments, deciding each turn whether to **clarify, recommend, refine, compare, or refuse** — over a stateless API (full history every call), within an 8-turn / 30-second budget. Two facts shaped every design choice: (1) the provided catalog (377 Individual Test Solutions) already contains 100% of the answers in the sample traces, so this is a *retrieval + grounding* problem, not a knowledge problem; and (2) the replay harness ends a conversation as soon as a shortlist appears, so the **first committed shortlist must be complete**.

Stack & why

- **FastAPI + Pydantic** — the schema is non-negotiable and scored on every response, so I model it exactly and keep the *request* side lenient (defaults + ignore-unknowns + validation handler) so a malformed call can never produce a 422/500 that fails schema compliance.
- **OpenAI gpt-4o-mini** for chat and **text-embedding-3-small** for embeddings — one key for both, strong structured-output support (`response_format=json_schema`, which *guarantees* a schema-valid action), fast and cheap (whole eval costs a few cents). The PDF explicitly allows "raw OpenAI SDKs". The LLM client is provider-agnostic (OpenAI-compatible base URL), so Groq / Gemini / OpenRouter are a one-line swap.
- **BM25 (rank_bm25) + NumPy cosine** — no vector DB needed for 377 items; embeddings are precomputed offline and committed, so the deployed service does zero build-time API calls and loads instantly.

Retrieval

Hybrid, fused with **Reciprocal Rank Fusion** (rank-based, so no score normalisation): - **BM25 (lexical)** nails exact skill/product tokens — `docker`, `hipaa`, `opq32r`, `sql`. - **Dense cosine** catches fuzzy intent — "reliability / dependability" → DSI, "customer calls" → contact-centre simulations. Two pieces of **domain knowledge** are encoded rather than left to chance: - **Alias expansion** for abbreviations/jargon (OPQ, GSA, SJT, JS, "cognitive"→ability/aptitude...) to lift lexical recall. - **Staple injection**: cross-cutting instruments (OPQ32r personality, Verify G+ cognitive, Graduate Scenarios SJT, DSI safety) are *always* added to the candidate pool. A skill-only query like "Java developer" would never surface a personality test by similarity, yet the sample batteries almost always include one — so the agent must at least be *able* to choose it. The query is the **cumulative user intent** (all user turns, latest emphasised); because the API is stateless, mid-conversation edits like "add AWS / drop REST" are part of the query and the right candidates surface with no server state.

Agent & prompt design

One structured LLM call per turn over (*system rules + retrieved candidates + full history*) returns `{action, reply, recommended_ids, end_of_conversation}`. The **candidate list is the only source of truth**; the model picks ids and the **server maps ids → exact catalog name/url/test_type**, so hallucinated URLs are structurally impossible. The prompt encodes the decision policy directly (the behaviours the probes test): clarify only when a *pivotal* unknown exists (rich queries are answered immediately); recommend a *complete battery* (skill tests + cognitive + personality + SJT/sim as the role warrants); refine by re-deriving the full list and applying add/drop/replace exactly; compare using only candidate descriptions; refuse legal / off-topic / prompt-injection. `test_type` is derived from the catalog `keys` via SHL's A/B/C/D/E/K/P/S legend.

Evaluation

Three harnesses, all runnable locally: - **Recall@10** with an **LLM user-simulator** that mirrors SHL's replay (answers from persona facts, "no preference" otherwise, ends on a shortlist). Scores the first committed shortlist vs the labelled final shortlist. **Mean Recall@10 = 0.67** across the 10 public traces - three runs against the *deployed* endpoint gave 0.65 / 0.67 / 0.69 (deterministic replay: 0.61-0.63). - **Retrieval-ceiling** diagnostic (Recall@50 of the retriever alone) = **0.81**, which localised the gap to *composition* rather than retrieval and told me where to spend effort. - **Behavior probes** (binary assertions): no-rec-on-vague-turn-1, rich-query-recommends, refuse off-topic / legal / injection, honours add & drop, grounded compare, all URLs in catalog. **Pass-rate = 9/9**. Plus 14 unit tests over the deterministic core (grounding, schema enforcement, `test_type` derivation). The harness can also score the **live** endpoint over HTTP.

What didn't work / trade-offs (and how I measured it)

Iterating against the harness took **Mean Recall@10 from 0.44 → ~0.67** (stable live range 0.65-0.69): - **Dense-only retrieval** missed staple instruments for skill-only queries (OPQ32r/Verify G+ absent from candidates) → added BM25 + always-injected staples: **0.44 → 0.56**. - **Minimal first shortlists** under-scored because the harness ends at the first shortlist → prompted for *complete batteries*, marked staples in-context, and encoded the SHL rule that OPQ reports imply the base OPQ32r: **0.56 → 0.61**. - **Dropped named skills / wrong near-duplicate variant** (e.g. "Docker" omitted, "SQL Server 2014" chosen over "SQL (New)") → a pre-finalise checklist + variant-coverage guidance: **0.61 → 0.69**. - **Over-clarifying** (one trace stalled 8 turns) → a hard 1–2 question clarification budget. - **Naive "recommend early"** failed the vague-turn-1 probe; making clarify-vs-recommend an explicit, example-driven policy fixed it without hurting rich-query recall. - **Restricting the model to candidate ids** trades a little recall (a relevant item outside top-K can't be chosen) for a hard no-hallucination guarantee — I judged groundedness the higher-value axis for this rubric and widened top-K to 40 + staples to recover recall. - Known data quirk: one report lists all 8 categories (scraping artifact); `test_type` derivation is mechanical and doesn't affect Recall (scored on item identity).

AI tools used

Built with **Claude Code** (Anthropic) as the coding agent for scaffolding, the retrieval/agent implementation, and the eval harness; all design decisions, the retrieval strategy, prompt policy, and evaluation were directed and reviewed by me. Runtime model is OpenAI `gpt-4o-mini`.